Physics Programming Project Report

Simulation of an N-body system

Malaguti Tommaso Patarchi Davide

Last code update: 25 August 2026

1 Introduction and Project Topic

This project follows the proposed `N-bodies` assignment. Specifically, we developed a program able to simulate the evolution of a system of N bodies subject to gravitational interaction. Compared to the basic model, our implementation was extended to include collision between bodies, a graphical interface that shows the evolution of the system in real time, and a study of the conservation of momentum and angular momentum.

2 Design and Implementation Choices

To develop the program, we made the following design and implementation choices.

2.1 Code Organization

To ensure modularity and efficiency, we split the code into different files. The code structure and declarations are in the header file (`gravity.hpp`), while the definitions are in the source file (`gravity.cpp`). The main execution is handled separately in the file `main.cpp`, and there is also a dedicated test file (`gravity.test.cpp`).

The whole project is wrapped in a namespace called `nbodies`. Inside it, we defined a further namespace dedicated to physical constants, `constants`, which includes the universal gravitational constant G, the integration time step dt (set to one minute), and the astronomical unit AU.

2.2 The vector struct

To simplify vector calculations, we defined the `vector` struct, which represents a two-dimensional vector. We chose a struct instead of a class because there is no class invariant that needs to be guaranteed. The struct exposes the methods needed for vector operations: addition (`operator+`) and subtraction (`operator-`) between vectors and multiplication (`operator*`) and division (`operator/`) by a scalar. It also has an equality operator between vectors (`operator==`) and a function to compute the norm of the vector(`norm`). These features were implemented by overloading the operators `+`, `-`, `*`, `/`, `==`.

2.3 The Planet class

The `Planet` class represents a planet object and is defined by the following attributes:

- a mass (`double mass_`), which is a class invariant checked by the `check_mass` method;

- an `nbodies::vector` that defines its position (`position_`);
- an `nbodies::vector` that defines its velocity (`velocity_`);
- an `nbodies::vector` that defines its acceleration (`acceleration_`);
- a radius (`double radius_`), also a class invariant, checked by the `check_radius` method.

The class methods are the constructor `Planet` and the getter methods, which give access to the attribute values, even though the attributes are declared in the private section of the class. The getter methods exist in both const and non-const versions.

2.4 The System class

The `System` class represents the system of planets and is defined by an `std::vector` of `nbodies::Planet` objects, declared in the private section. The class methods are:

- `get_planet`, to access each planet stored in the vector;
- `size`, which returns the size of the vector;
- `iteration`, which updates the state of the system at each simulation step;
- `collision`, which detects and handles collisions between planets;
- `read_file`, needed to read input data from a file;
- `energy` and `scale_energy`, to compute and check energy conservation;
- `momentum` and `scale_momentum`, to compute and check momentum conservation;
- `angular_momentum` and `scale_angular_momentum`, to compute and check angular momentum conservation.

The iteration function

The `iteration` function was implemented following this logic:

- **Position:** to update the position of each planet, we used the `std::for_each` algorithm, which updates the position of every planet in `nbodies::System` using formula (1).

$$\vec{s}_i(t+\Delta t) = \vec{s}_i(t) + \vec{v}_i(t)\Delta t + \frac{1}{2}\vec{a}_i(t)\Delta t^2, \tag{1}$$

- **Acceleration:** since computing the new velocities also requires the initial acceleration values, these were saved in an `std::vector init_acc` using the `std::transform` algorithm. Then, using a `for` loop, each acceleration was updated using formula (2).

$$\vec{a}_i(t+\Delta t) = -\sum_{j=1\, j\neq i}^{N} G\frac{m_j}{(|\vec{s}_j(t+\Delta t) - \vec{s}_i(t+\Delta t)|)^3}(\vec{s}_i(t+\Delta t) - \vec{s}_j(t+\Delta t)), \tag{2}$$

- **Velocity:** to update the velocities, we instead used a `for` loop that goes through the `std::vector` and updates the velocity of each `Planet` using formula (3).

$$\vec{v}_i(t+\Delta t) = \vec{v}_i(t) + \frac{1}{2}\left(\vec{a}_i(t) + \vec{a}_i(t+\Delta t)\right)\Delta t \tag{3}$$

The collision function

Collisions between planets are modeled as inelastic collisions: when the distance between two planets becomes smaller than the sum of their radii, they merge into a single planet. The mass of this new planet equals the sum of the masses of the two original planets, and its position and velocity are given by formula (4) and formula (5).

$$\vec{s}_{\text{new}} = \frac{m_i \vec{s}_i + m_j \vec{s}_j}{m_i + m_j} \tag{4}$$

$$\vec{v}_{\text{new}} = \frac{m_i \vec{v}_i + m_j \vec{v}_j}{m_i + m_j} \tag{5}$$

As for the radius of the newly formed planet, it follows formula (6).

$$r_{\text{new}} = \sqrt{r_1^2 + r_2^2} \tag{6}$$

The collision function handles collisions using two nested `for` loops, which work as follows:

The outer loop uses index i to go through `nbodies::System`. The loop's exit condition checks the size of the system at every step, since it can change because of collisions. At each iteration, the initial size `init_size` is saved. The inner loop uses index j, starting from $i+1$. This makes sure that every unordered pair (i, j) is checked only once, and prevents a planet from colliding with itself.

The `if` statement checks whether the physical radii of the bodies overlap. If a collision happens: the new physical quantities are computed (position and velocity of the center of mass, total mass, combined radius), and the original bodies are removed from the `std::vector` using the `erase` method. Removal starts from the higher index (j) so that the position of the lower index (i) is not affected. The new planet is then created, its gravitational force from the other bodies in the system is computed, and it is added at the end of the vector with `push_back`. Finally, the inner loop is stopped with `break`.

If `planets.size() < init_size`, it means that a collision has changed the structure of the vector, so index `i` is reset to 0 to restart the check on the updated vector. Otherwise, the loop continues with `++i`.

The read_file method

The `read_file` method is used to read the planet data from the data file and insert it into an `nbodies::System` object using `push_back`. The acceleration of each planet is computed from the data of the other planets.

Energy conservation

There are two methods used to study energy conservation.

The `energy` method works as follows: the total kinetic energy is computed with an `accumulate` algorithm, which adds up the kinetic energy of each body to get the total kinetic energy. We used formula. (7)

$$K = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2 \tag{7}$$

The potential energy is computed using an outer `for` loop (index i) together with an inner `std::accumulate`. The inner part adds up the gravitational contributions between bodies using formula (8).

$$U = -\sum_{i<j} G \frac{m_i m_j}{|\vec{s}_i - \vec{s}_j|} \tag{8}$$

The upper limit of the `std::accumulate` algorithm for the potential energy is `planets.begin() + int(i)`. This means that the inner loop only considers the interactions between planet i and the planets j that come before it in the list ($j < i$).

The second method, `scale_energy`, adds up the absolute values of the kinetic and potential energy contributions, giving a reference value (a "scale") used to compare the energy variation during the simulation. This function is important because it helps to understand whether a variation of several orders of magnitude should be considered significant or not.

Momentum conservation

The `momentum` and `scale_momentum` methods are used to study momentum conservation.

The `momentum` method uses two `accumulate` algorithms to compute the x and y components of the momentum, and returns an `nbodies::vector` containing the momentum value. We used formula (9).

$$\vec{P} = \sum_{i=1}^{N} m_i \vec{v}_i. \tag{9}$$

The `scale_momentum` method instead computes the momentum value as an `nbodies::vector`, adding the absolute values of the momenta of the individual planets. Just like `scale_energy`, this function gives a reference scale used to study the variation of momentum.

Angular momentum conservation

The `angular_momentum` and `scale_angular_momentum` methods are used to study the conservation of angular momentum.

The `angular_momentum` method computes the angular momentum value using the `accumulate` algorithm, following formula (10).

$$L_z = \sum_{i=1}^{N} m_i (S_{i,x} v_{i,y} - S_{i,y} v_{i,x}) \tag{10}$$

The `scale_angular_momentum` method also computes the angular momentum value, but adds the absolute values of the angular momenta of the individual planets. It plays the same role as `scale_energy` and `scale_momentum`.

2.5 Unit Conversion

Finally, we implemented the functions `AU_pixel_x`, `AU_pixel_y` and `pixel_radius`, to proportionally convert values expressed in astronomical units into pixels. These functions are necessary because the datasets we use define the planet positions on a Cartesian plane with a width of two astronomical units (from -1 to 1 on both the x and y axes), while, for display through the SFML library, these positions need to be translated into the reference system of an 800x800 pixel window. For the conversion along the x axis and y axis we used formula (11) and formula (12).

$$x_{\text{pixel}} = 400 \cdot \left(1 + 0.9 \cdot \frac{s_x}{\text{AU}}\right) \tag{11}$$

$$y_{\text{pixel}} = 400 \cdot \left(1 - 0.9 \cdot \frac{s_y}{\text{AU}}\right) \tag{12}$$

The `pixel_radius` function takes the radius value `r` as input and returns a `float` value calculated according to the following definition (13).

$$\text{pixel_radius}(r) = \begin{cases} 1.0 & \text{if } r < 10 \\ \lfloor \log_{10}(r) \rfloor & \text{if } r \geq 10 \text{ and } \frac{r}{10^{\lfloor \log_{10}(r) \rfloor}} < 5 \\ \lfloor \log_{10}(r) \rfloor + 1 & \text{if } r \geq 10 \text{ and } \frac{r}{10^{\lfloor \log_{10}(r) \rfloor}} \geq 5 \end{cases} \tag{13}$$

As highlighted in Equation (13), the algorithm estimates the order of magnitude of r, rounding up the exponent when the base meets or exceeds the threshold of 5.

3 How to Run the Program

The configurations we created can be found in the `data/` directory, and are shown in Figure 1.

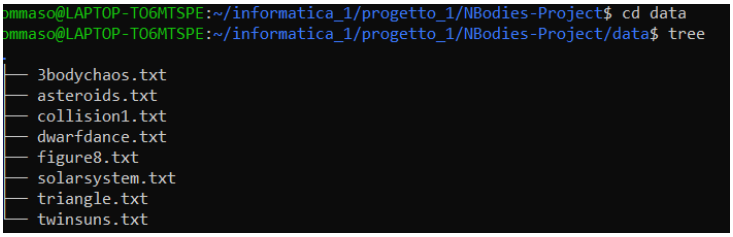


Figure 1: Names of the different data files

To choose which configuration to simulate, you need to enter the name of the desired data file inside the `read_file` function in `main.cpp`, at line 20, as shown in Figure 2.

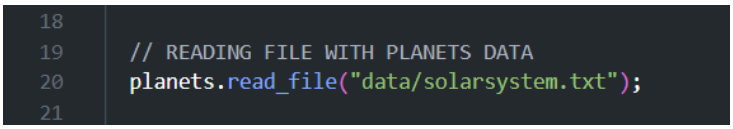


Figure 2: Code line where to name the data file

Input Parameters and Output Format

- **Input**: For each planet, the data file must contain the mass, the x and y position, the x and y velocity, and the radius (in this order).
- **Output**: The program's output is both numerical and graphical. Once the program starts, the user is asked how many days the simulation should last. After the answer is given, a graphical window opens showing the evolution of the planets over time. Once this window is closed, the terminal shows the values of the total energy, momentum, and angular momentum, and whether they were conserved or not.

4 Interpretation of the Results

After running, the program prints several pieces of information about energy, momentum, and angular momentum:

- **Initial and final value**: energy/momentum/angular momentum of the system computed at the beginning and at the end of the bodies' evolution.
- **Scale values**: energy/momentum/angular momentum of the system computed without allowing possible cancellations between the different bodies, obtained by summing the absolute values of the quantities.
- **Relative error**: if the initial and final values are not *exactly* the same number, the relative error is computed with formula (14).

$$\frac{X_{final} - X_{initial}}{X_{average-scale}} \tag{14}$$

where $X_{average-scale}$ is the average between the initial and final scale values. If the relative error is smaller than 1%, the quantity is considered conserved.

The goal is to show that momentum and angular momentum are always conserved, while energy is conserved only when there are no inelastic collisions.

5 Testing Strategy

To check the robustness of the code, several tests were carried out:

5.1 Vector tests

- **Operator** $+$: addition with an `nbodies::vector` with positive coordinates, zero coordinates and negative coordinates;
- **Operator** $-$: subtraction with an `nbodies::vector` with positive coordinates, zero coordinates and negative coordinates;
- **Operator** $*$: multiplication by a positive scalar, a negative scalar and 0;
- **Operator** /: division by a positive scalar, a negative scalar and a check that `throw std::invalid_argument` works correctly for division by 0;
- **Operator** $==$: comparison with an identical `nbodies::vector` and with two `nbodies::vector` that share only one identical coordinate;
- **norm**: norm of an `nbodies::vector` with positive, negative and zero coordinates;

5.2 Planet tests

- **Constructor and getter functions**: creation of a planet through the constructor and check of the assigned values using the getter functions (`mass`, `position`, `velocity`, `acceleration`, `radius`);
- **Mass and radius exceptions**: check that `throw std::invalid_argument` works correctly when the mass or the radius are not positive, or when the value is `NaN`;
- **Non-const getter functions**: check that the getter functions, when called on a non-const `nbodies::Planet`, work correctly even after the object has been modified;

5.3 System tests

- **Size exception**: check that `check_size()` works correctly when the size of `nbodies::System` is 0;
- **read_file, size, get_planet**: reading of a data file with the `read_file` function. Then, checking the size of `nbodies::System` with the `size` method and the values assigned to each planet, using the `get_planet` method to access the i-th planet and the previously tested getter functions to retrieve its individual quantities. Finally, checking that `throw std::runtime_error` works correctly when the data file cannot be opened correctly, or when data are the wrong type;
- **iteration**: simulation of the evolution of a system made of a single body moving in a straight line at constant speed and of a system made of two bodies initially at rest on the x axis;

- **collision**: simulation of the collision of a two-body system and of a three-body system that never collides;
- **energy, momentum, angular_momentum**: computation and comparison of the values of energy, momentum and angular momentum (including their scale values) for one-body and two-body systems;

5.4 Tests of the SFML functions

- **AU_pixel_x and AU_pixel_y**: check that the conversion from astronomical units to pixels is correct;
- **pixel_radius**: check of the conversion from meters to pixels, evaluating both rounding down and rounding up;

6 Use of Generative Artificial Intelligence

To help find possible flaws in the program and identify all the necessary tests, in order to guarantee the correct functioning of the project, we used the generative artificial intelligence system Claude. This system also generated reasonable and working data files, in order to obtain interesting graphical simulations. Nevertheless, the AI's work was always carefully supervised by both students and used only when it was really necessary.